

THE DESIGN GRAMMAR ONTOLOGY: A SEMANTIC CORE FOR CROSS-PLATFORM DESIGN-INTENT MANAGEMENT AND DATA-DRIVEN DESIGN VALIDATION

SUBMITTED:

REVISED:

PUBLISHED:

EDITOR:

DOI:

Evgenii Ermolenko, PhD Candidate,

University of Minho; EAAD; Lab2PT; ISISE, Guimaraes, Portugal

ORCID: 0009-0003-1228-9550

email: erarchitech@gmail.com

Bruno Figueiredo, Pr,

University of Minho; EAAD; Lab2PT; IN2PAST, Guimaraes, Portugal

ORCID: 0000-0001-8439-7065

email: bfigueiredo@eaad.uminho.pt

Miguel Azenha, Pr,

University of Minho; ISISE; ARISE; Department of Civil Engineering, Guimaraes, Portugal

ORCID: 0000-0003-1374-9427

email: miguel.azenha@civil.uminho.pt

SUMMARY: Design rules and the intent behind them fragment across files, tools, and disciplines: each handover exchanges only final geometry, and the rules that produced it are lost. Large language models now interpret regulations reliably, but the interpreted rule has nowhere durable to live: inside a prompt-response cycle it cannot be audited, versioned, or re-executed deterministically. This paper presents the Design Grammar (DG) ontology, developed through explicit ontology-engineering activities with defined competency questions and systematic reuse of established design and topology ontologies. The schema organises an object-centred FBS core over five modular graph layers sharing one compact core, so further layers can be added without disturbing it. Function, the design intent an object must fulfil, is externalised into the graph, defined by shared vocabulary, and enforced as decomposed SWRL rules under an auditable violation-pattern encoding — a rule fires only when its constraint is broken — complemented by OWL 2 DL and SHACL reasoning over an explicit RDF projection of the property graph. Design objects act as identity carriers, so separate platform representations of one conceptual object resolve to a single identity. Parametric configurations persist as time-stamped Design States, and a provider-agnostic, human-in-the-loop LLM workflow operationalises rule authoring. Four competency questions are answered against the running graph, with query and result shown for three; the fourth is flagged honestly as implemented but not yet data-populated.

KEYWORDS: Design Grammars; Ontology; Knowledge Graph; SWRL; Design Intent; openBIM; Compliance

REFERENCE: Evgenii Ermolenko, Bruno Figueiredo & Miguel Azenha (2026). *The Design Grammar ontology: a semantic core for cross-platform design-intent management and data-driven design validation*. *Journal of Information Technology in Construction (ITcon)*, Vol. 31, pg. 1-xx, DOI: 10.36680/j.itcon.2026.xxx

COPYRIGHT: © 2026 Evgenii Ermolenko, Bruno Figueiredo, Miguel Azenha. This is an open access article distributed under the terms of the Creative Commons Attribution 4.0 International (<https://creativecommons.org/licenses/by/4.0/>), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

1. INTRODUCTION

The built environment is governed by an increasingly dense body of regulatory codes, planning guidelines, and project-specific design rules that architects and engineers must navigate throughout the design process. Manual compliance validation is time-consuming, error-prone, and difficult to scale across large project portfolios and design stages. The emergence of Building Information Modelling (BIM) as the dominant digital medium for architectural design has created an opportunity to automate compliance checking by querying structured geometric and semantic data against machine-readable rule sets (Eastman et al., 2009).

Design grammars, first introduced by Stiny and Gips (1972) as shape grammars, provide a generative, rule-based formalism for describing spatial configurations, and subsequent extensions have brought the formalism closer to practical compliance-checking workflows (Duarte, 2005; Beirão et al., 2011). However, translating design-grammar rules into executable ontology representations that operate directly on BIM data remains largely unsolved. A central obstacle is the absence of a shared, well-structured ontology that can host such rules: existing efforts tend to author ontologies ad hoc for individual studies, with little attention to reuse, alignment, or systematic development. This paper addresses that gap by focusing on the ontology itself rather than on any particular software system built around it.

An ontology is not an active interpreter. It is the shared code space of a knowledge graph: the agreed vocabulary and axioms that any interpreting agent, human or machine, must follow when entities are classified, relationships are inferred, and queries are answered. Establishing such a shared conceptual foundation across the diverse domains of architectural knowledge is the precondition for genuine design-grammar reasoning. Ontology-based compliance checking has been studied in the AEC sector for two decades, from early rule systems operating on enriched IFC files (Eastman et al., 2009) to OWL-based approaches using IfcOWL, SPARQL, and SWRL (Pauwels & Terkaj, 2016; Pauwels et al., 2017), knowledge-graph backends for rule storage and evaluation (Zhou et al., 2022), and property-graph stores for heterogeneous building knowledge (Sacks et al., 2020). The present work builds on this lineage but inverts its emphasis: rather than building another validation system, it develops the common ontology that such systems presuppose. This schema underlies the cross-platform knowledge-management framework introduced in a companion paper (Ermolenko et al., 2026); Table 1 states precisely what each paper contributes so the two remain distinguishable.

Table 1: What appears in the companion paper (Ermolenko et al., 2026) versus this paper.

Scope	Companion (ptBIM 2026)	This paper
Primary subject	System architecture: communication, data, and analysis layers of the DG platform	The Design Grammar ontology: core, five graph layers, SWRL encoding
Content	Service topology and data flow; two case studies with validated outcomes	Object-centred FBS core; competency-question-driven schema evaluation; property-graph/OWL projection
Shared material	Same running system and the same 75 m height example, for continuity	Same running system; this paper defines the schema the companion's services operate on

Status at submission	Presented at ptBIM 2026	Submitted to ITcon
----------------------	-------------------------	--------------------

The most recent wave of automated compliance checking delegates the entire pipeline to large language models: LLM-centric frameworks interpret regulations, extract BIM data, execute checks, and generate reports with high rule-interpretation accuracy (Iversen & Huang, 2026; Madireddy et al., 2025; Chen et al., 2024). These systems demonstrate that natural-language rule interpretation is no longer the bottleneck; what remains unsolved is where the interpreted rule lives. A rule that exists only inside a prompt-response cycle cannot be audited atom-by-atom, versioned, shared across projects, or re-executed deterministically. The industry context sharpens the same requirement: reviews of semantic-web adoption identify the transition from file-based BIM maturity stage 2 to the object-level collaboration of stage 3 as the sector's central interoperability challenge and name ontologies as the key enabling technology (Palihakkara et al., 2025), while openBIM deliverables are increasingly mandated and are moving from pilot to production (buildingSMART, 2025, 2026) in a BIM software market converging on AI-assisted, model-centric workflows (Fortune Business Insights, 2026). The present work is complementary to the LLM wave: it supplies the persistent, queryable substrate in which machine-interpreted rules become durable, inspectable artifacts, while LLMs are confined to the roles they perform reliably — encoding, recognition, and consultation.

Against this background, the paper makes three contributions to data-driven design development.

- A systematic ontology-development methodology applied to design grammars — explicit specification, knowledge sources, competency questions, and implementation-grounded milestones — where previous design-grammar ontologies were authored ad hoc (section 2).
- The resulting schema: an object-centred core derived from the Function–Behaviour–Structure framework, spanning five modular knowledge-graph layers, with a violation-pattern SWRL encoding and typed cross-layer bridges. Because every layer maps onto the same compact core, the schema is scalable — further layers can be added without disturbing existing ones — and it is packaged as a vendor-neutral core with modular alignment extensions to external vocabularies (section 3).
- Lifecycle operability: parametric configurations of design objects are captured as time-stamped Design States for longitudinal validation; design objects act as identity carriers, so separated representations of one conceptual object in different authoring platforms resolve to a single identity and share unique semantic properties; and an AI-facilitation layer — a provider-agnostic LLM gateway whose context assembler compiles the graph's vocabulary, rule conventions, and query templates into each model call — lets practitioners author rules and serialise parametric models under human confirmation, without formal ontology expertise (section 4).

The remainder of the paper is organised as follows. Section 2 presents the ontology-development methodology. Section 3 describes the ontology: its core, graph layers, Design State, rule encoding, and property-graph/OWL projection. Section 4 presents lifecycle integration and the AI-facilitated workflow. Section 5 evaluates the ontology by answering its competency questions directly against the running graph. Section 6 discusses expressiveness, positioning, and scalability. Section 7 concludes.

2. ONTOLOGY DEVELOPMENT METHODOLOGY

2.1 Development process and grounding

There is no single correct way to model a domain; viable alternatives always exist (Noy & McGuinness, 2001). The design-grammar ontology was therefore developed through a structured set of ontology-engineering activities organised as an iterative process across increasing levels of formality (Figure 1), combining the scenario-based development process of the NeOn methodology (Suárez-Figueroa et al., 2015) with the lighter-weight, industrial-oriented practices of the LOT framework (Poveda-Villalón et al., 2022), following the general activity model taught in Sack (2019). The specification establishes the definition and boundaries of the ontology. Conceptualisation structures domain knowledge into a conceptual model. Formalisation expresses that model in OWL 2 constructs and SWRL Horn clauses. Implementation realises it as a property graph in Neo4j, authored in a dedicated ontology editor (Protégé) and checked by the deterministic reasoning services of sections 3.4 and 3.6. These principal activities are accompanied by continuous supporting activities: knowledge acquisition, evaluation, reuse, documentation, and configuration management. Evaluation combines the competency

questions of section 2.2, answered directly against the running graph in section 5; a structural self-audit against the ontology pitfall catalogue of Poveda-Villalón et al. (2014), reported in section 5.4; and machine-checked OWL 2 DL consistency, also in section 5.4.

Because the ontology is co-developed with a running system, each formalisation step is grounded in an implementation checkpoint (Figure 1): the runtime graph encoding of rules and validation runs came first, followed by the cross-cutting core band and the knowledge layer, the vendor-neutral core with the three-part Design State, the OWL 2 DL and SHACL reasoning services, the serialisation of live parametric canvases with the cross-platform identity registry, and finally script-level intelligence — the five graph layers and the core band named here are defined in Table 4 (section 3.2). Each checkpoint was verified against the competency questions and regression-tested before the next was undertaken, implementing the continuous-evaluation principle of the methodology. The ontology is designed to operate within mainstream BIM and parametric-design environments — BIM authoring tools (e.g., Revit), geometry modellers (e.g., Rhinoceros), and visual-programming editors (e.g., Grasshopper, Dynamo); the surrounding software services are described only insofar as they ground the schema's runtime (section 4).

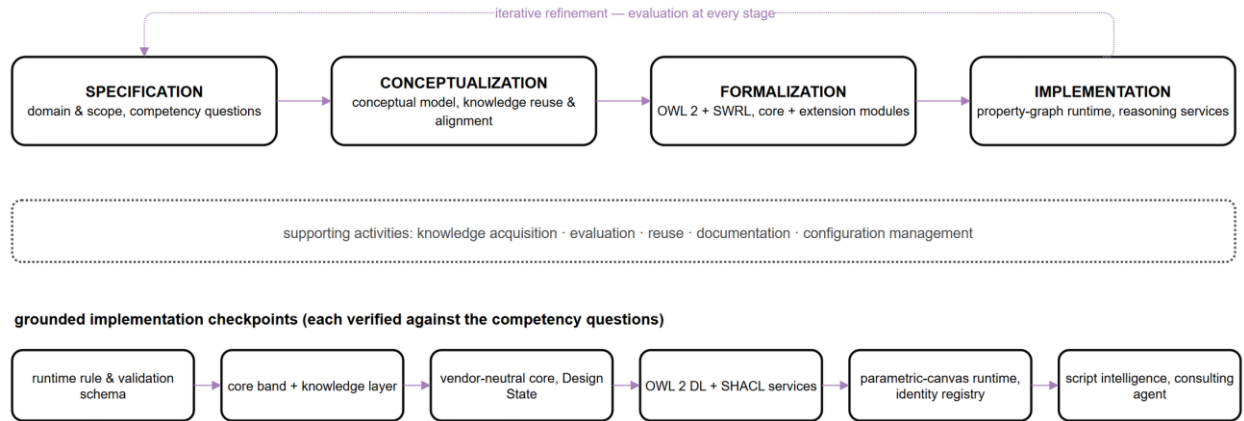


Figure 1: Ontology-development activities (top), continuous supporting activities (middle), and the grounded implementation checkpoints of the DG ontology (bottom).

2.2 Specification: purpose, domain, scope, and competency questions

The specification fixes the purpose, domain, and users of the ontology. Its purpose is to represent design grammars as durable, auditable knowledge that can be authored in natural language, evaluated against evolving states of the design, and reused across disciplines and projects. Its domain is data-driven parametric design, spanning RIBA Plan of Work Stages 1 through 3 — from preparation and briefing through concept design to spatial coordination (RIBA, 2020) — and extending into the detailing and validation activities that consume the rules authored earlier. The intended end-users are architects, engineers, computational designers, and clients, while maintainers are BIM departments. The anticipated benefits are rigorous communication of design grammars, simplified management of multiple constraints, automatic design-intent validation, and durable design knowledge that survives reformulation and reuse.

The scope is informed by how architects reason: through problem–solution co-evolution (Schön, 1992; Poon & Maher, 1997; Dorst & Cross, 2001), framing and abductive reasoning (Dorst, 2011), analogical and case-based reasoning (Gero, 1992; Mubarak, 2004; Fu et al., 2015), and function–structure mapping and decomposition (Gero, 1990). From these, the ontology is required to support the decomposition of a design problem into computable parts, the modular structuring of design knowledge, interoperability and shared perception across stakeholders, the translation of design knowledge among authoring systems, and semantic reasoning with automatic design-intent validation based on defined rules.

Feasibility is bounded by the competency questions the ontology must answer, which double as acceptance criteria for evaluation (Table 2). Section 5 answers CQ1 through CQ4 directly against the running graph.

Table 2: Competency questions and where each is answered.

#	Competency question	Answered in
---	---------------------	-------------

CQ1	Can a design rule authored in natural language be decomposed into individually addressable, auditable parts and grounded in shared vocabulary?	§5.1
CQ2	Does the current design state of a scheme satisfy the constraints defined for it, with an individually addressable verdict per object?	§5.2
CQ3	Can a planning-constraint rule be traced to the specific parametric quantity it constrains?	§5.3
CQ4	Is the schema itself terminologically consistent and structurally complete under its own integrity constraints?	§5.4

2.3 Knowledge sources, reuse and alignment

A core principle of the methodology is reuse of existing ontologies rather than de novo authoring. Three source families were analysed for versatility of implementation, accessibility for external use, and maintainability, and were aligned to the design-grammar domain under a common SWRL convention (Table 3). Reuse never couples the vendor-neutral core to a particular external vocabulary: BOT and Topologic are bridged through dedicated extension modules rather than embedded in the schema, following the core-plus-extension modularisation established for W3C vocabularies such as SOSA/SSN (Janowicz et al., 2019), and the same mechanism accommodates further alignments as the ecosystem around the ontology grows.

Table 3: Reuse and alignment of the three source families.

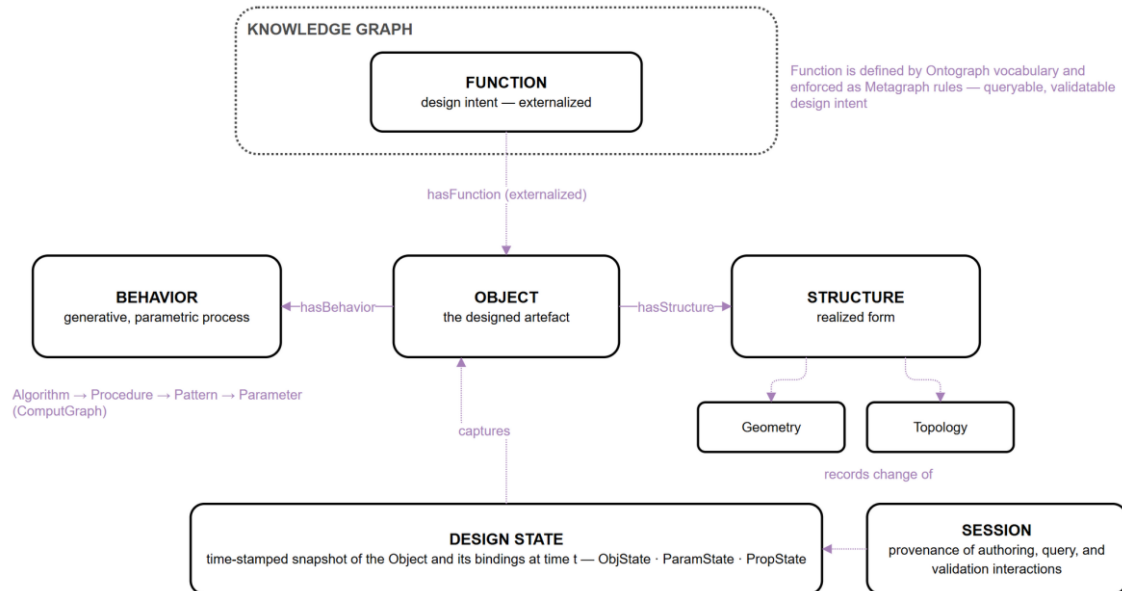
Source	What was taken	What was adapted	What was left out
FBS (Gero, 1990; Gero & Kannengiesser, 2004)	The Function–Behaviour–Structure decomposition as the core band’s conceptual scaffolding	Function relocated from an object attribute to an externalised, graph-resident construct (section 3.1)	The full situated-FBS process model (the formulation–reformulation cycle); only the three-way structural decomposition is retained
DCM (Lin, 2016)	The semantic/topological/geometric information split, and criteria expressed as SWRL Body→Head clauses over computed predicates	Generalised from architectural-design criteria specifically to any quantitative design constraint	DCM’s own topology-predicate library; topological relations route through the BOT/Topologic extensions instead
BOT (Rasmussen et al., 2020)	Building-topology relationships for spatial adjacency and containment	Bridged through an extension module rather than imported into the core	BOT’s full building-element taxonomy; only the topological relations are aligned
Topologic (Jabi & Chatzivasileiadi, 2021)	Non-manifold topological representation for spatial reasoning	Bridged through the same extension mechanism as BOT	The computational topology engine itself; only its conceptual relations are aligned

3. THE DESIGN GRAMMAR ONTOLOGY

This section presents the resulting ontology: the object-centred FBS core (section 3.1), the five graph layers, their modular packaging, and the schema’s element inventory (section 3.2), the Design State (section 3.3), the SWRL encoding and reasoning services (section 3.4), the mapping of parametric definitions (section 3.5), and the property-graph/OWL projection that makes the schema’s OWL semantics operational (section 3.6). Table 4 gives the normative vocabulary — every layer name, its role, and its element counts — in one place; the complete schema, its node labels, relationship types, and full class and property inventory, is maintained as a versioned, open ontology project in the accompanying repository [reference to be added upon publication].

The conceptual heart of the ontology is an object-centred rendering of Gero's Function–Behaviour–Structure triad (Gero, 1990; Gero & Kannengiesser, 2004): a cross-cutting band that sits above the graph layers and is referenced by all of them (Figure 2). The Object — the designed artefact — stands at the centre of the model, and the familiar FBS roles are arranged around it, with one deliberate repositioning. An Object carries Behavior, the generative, parametric process that produces it, and Structure, the realised geometric and topological form. Function, the design intent the artefact must fulfil, is not modelled as an internal attribute of the object: it is externalised into the knowledge graph, where it is defined by the vocabulary layer and enforced as decomposed rules.

Design intent thereby becomes a first-class, queryable, and validatable artifact rather than an implicit property of geometry — the precondition for communicating intent across disciplines and platforms, and for the persistent intent management that file-based exchange cannot provide (Wyke & Lindhard, 2025). Geometry and Topology complete the structural side of the core, so that the validated model and the parametric definition share one spatial vocabulary. Two further constructs close the band: the Design State, a time-stamped snapshot of an Object and its bindings (section 3.3), and the Session, a provenance record of every authoring, query, or validation interaction. Keeping these over-layer concepts in one band is what lets the graph layers remain decoupled yet interoperable.



3.2 Five graph layers, modular packaging, and project isolation

6

logically by a graph property and isolated per project by a project property (Figure 3, Table 4). Three typed cross-layer bridges make the layers mutually addressable: REFERS_TO grounds every Metagraph atom in Ontograph vocabulary; ATTRIBUTE_OF links a rule atom to the ComputGraph parameter it constrains; and VALIDATES connects an evaluated rule set to the object instances it checks. Because every layer references the shared core band rather than the internals of other layers, the layer set is open-ended: an additional layer — for cost or environmental-analysis knowledge, for example — can be mapped onto the same core without disturbing the existing schema.

Figure 3: The core band over the five coplanar graph layers, with the typed cross-layer bridges REFERS_TO, ATTRIBUTE_OF, and VALIDATES.

Table 4: The five graph layers plus the cross-cutting core band — role, principal node types, and element counts (core module v7; counts exclude extension modules).

Layer	Role	Principal node types	Cls	ObjP	DataP
Core + Ontograph	Cross-cutting FBS concepts (Core) plus the domain vocabulary every rule and function is expressed in (Ontograph)	Object, Function, Behavior, Structure, Geometry, Topology, DesignState, Session; Class, DataProperty, ObjProperty	18	10	9
Metagraph	Rule decomposition: every SWRL clause as individually addressable nodes	Rule, Atom, Var, Literal, Builtin	15	8	18
ComputGraph	Parametric process: the visual-programming definition	Algorithm, Procedure, Pattern, Parameter, Interface	13	10	7
ValidGraph	Validation evidence: runs, per-object results, integration config, reasoner	Run, ValidationEntity, ObjectInstance, Reasoner	12	13	26
SpecGraph	Accumulated project memory: notes, tags, conventions	SpecNote, SpecTag, SpecClass	4	2	8
Total (core module)			62	43	68

The schema is delivered as a compact, vendor-neutral core module accompanied by optional extension modules, following the core-plus-extension pattern (Janowicz et al., 2019). The core contains only design-grammar-native content and declares no external imports, so it loads cleanly, reasons quickly, and names no product or format. Extension modules carry the alignment axioms to external vocabularies — currently the W3C/OGC standards stack, building topology, and a non-manifold spatial hierarchy — and further extensions can be added in the same manner. Extensions engage automatically: when new entities are ingested into a given graph layer, the extension aligned with that layer allocates them to the proper external classes, so a consumer loads the lean core for schema editing and rule authoring and gains cross-vocabulary reasoning only where it is needed. All nodes carry a project property that scopes them to a named design project, allowing a single graph to host multiple concurrent projects with logical isolation while still permitting cross-project queries — lessons learned accumulate in a shared stakeholder memory rather than fragmenting across isolated databases. This isolation strategy is adequate for pilot-scale use — a single organisation operating a handful of concurrent projects on one graph instance; multi-tenant production scenarios may require sharding or per-project databases (section 6.2).

3.3 Design State: the unit of lifecycle traceability

A Design State is the ontology's unit of lifecycle traceability: a time-stamped, immutable snapshot of a design object and everything that determines it at a given moment (Figure 4). It composes three state kinds. The

ObjState binds the object and a neutral reference to its geometry; the ParamState captures the parameter bindings of the parametric definition — the sliders, toggles, and value inputs of the Behavior; and the PropState records, per object, the rule, the data property it constrains, and the observed value. A validation Run consumes such a snapshot and returns one Boolean verdict per object state — so per-object outcomes remain addressable — together with a flag confirming publication of colour-coded results to an external 3D viewer. Because snapshots are persisted rather than overwritten, the graph accumulates an ordered series DS_t0 ... DS_tn that can be compared, replayed, and selectively reinstated. Validation thereby turns from a point-in-time gate into a longitudinal record of how a scheme evolves across concept, parametric development, BIM detailing, and operation.

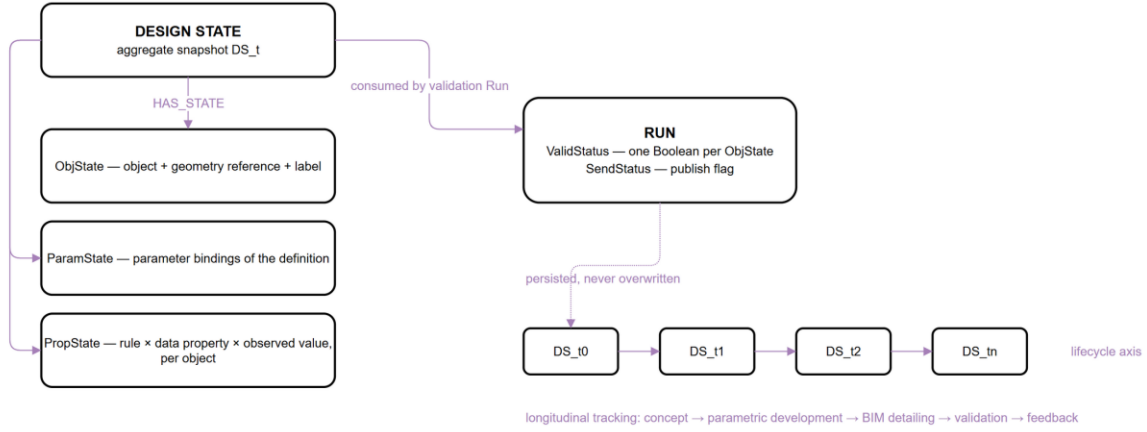


Figure 4: Design State composition (ObjState, ParamState, PropState), its consumption by a validation Run, and longitudinal tracking along the lifecycle axis.

3.4 SWRL rule encoding and deterministic reasoning services

Design rules are encoded as SWRL Horn clauses of the form Body $\rightarrow$ Head, where the body atoms express the condition that violates the constraint and the head atom asserts a boolean violation flag. This inverted, violation-first semantics simplifies rule evaluation: a rule fires if and only if the constraint is broken, avoiding the need to reason over the complement of a positive condition. For example, the regulation that maximum building height is 75 meters is encoded as:

Building(?b) $\wedge$ hasHeightM(?b, ?h) $\wedge$ swrlb:greaterThan(?h, 75.0) $\rightarrow$ violatesMaxHeight(?b, true)

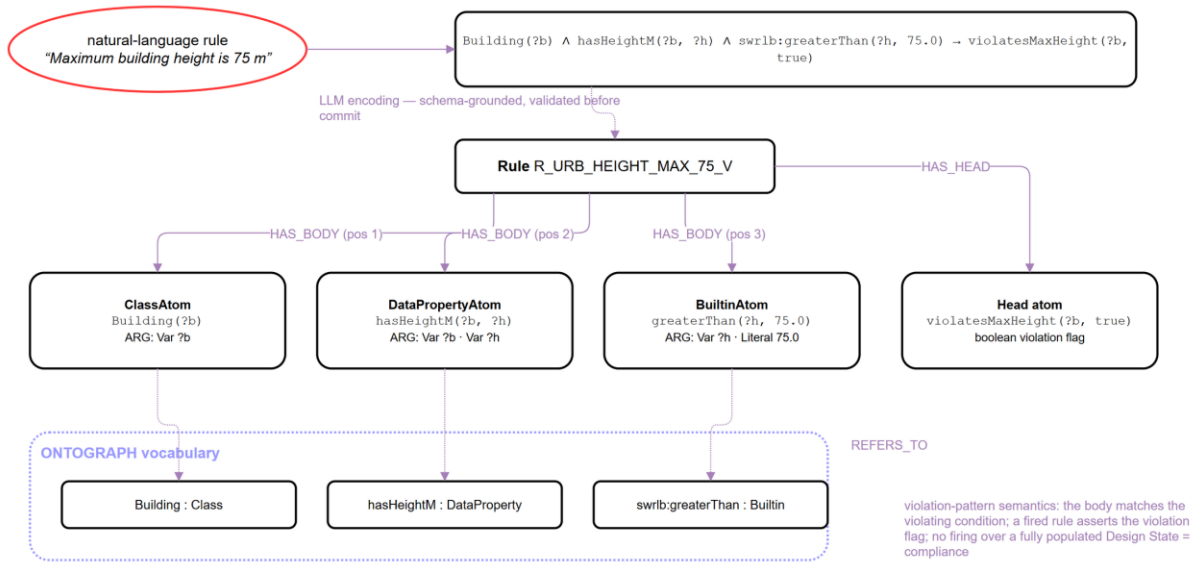


Figure 5: Anatomy of a violation-pattern SWRL rule: LLM encoding of a natural-language constraint, Metagraph decomposition into typed atoms with ordered arguments, and grounding of every atom in Ontograph vocabulary.

The complementary lower-bound pattern is illustrated by the constraint that buildings keep at least 12 metres of separation:

$\text{Building}(\text{?b}) \wedge \text{hasDistanceM}(\text{?b}, \text{?d}) \wedge \text{swrlb:lessThan}(\text{?d}, 12.0) \rightarrow \text{violatesMinDistance}(\text{?b}, \text{true})$

The two rules share the same class atom and differ only in the comparison builtin and the head predicate: this is exactly the pair a true range constraint decomposes into when a single property is bounded both above and below, preserving the principle that each rule node encodes one atomic constraint. Minimum constraints use `swrlb:lessThan`, maximum constraints `swrlb:greaterThan`, equality constraints `swrlb:notEqual`, and range constraints decompose into this lower/upper pair rather than a single compound rule, since compound rules are difficult to trace during debugging and audit. Each rule carries a structured identifier (e.g., `R_URB_HEIGHT_MAX_75_V`), is decomposed in the Metagraph into body and head atoms with ordered argument links to variables and literals, and is grounded in the Ontograph through `REFERS_TO` (Figure 5); Figure 7 in section 5.4 shows the minimum-distance rule's full decomposition with every atom, argument, and grounding populated from the running graph.

The SWRL validator is one of three deterministic reasoning services that share the graph under an explicit rule-partition policy assigning each rule category to exactly one owner. Quantitative threshold constraints on design states are evaluated by the SWRL validator. Terminological consistency of the ontology itself — class disjointness, property domains and ranges — is checked by an OWL 2 DL reasoner (HermiT). Structural and data-integrity constraints — required properties, cardinalities, enumerated kinds — are validated by SHACL shapes over an RDF projection of the graph (section 3.6), following the closed-world validation semantics for which SHACL is designed and which has proven effective for cross-domain building data (Hagedorn et al., 2023). All three services run in an isolated reasoner sidecar — a separate service process that assembles the RDF projection and runs the OWL/SHACL engines without holding a lock on the graph database — and their verdicts are persisted to the ValidGraph beside the design states they judge, so the evidence trail of every compliance claim (rule, state, reasoner, verdict) is reconstructible from the graph alone.

3.5 Mapping parametric definitions onto the ComputGraph

The ComputGraph is designed so that the parametric definitions architects already build in visual-programming environments can be expressed in the ontology without translation loss. A definition maps one-to-one onto the ComputGraph hierarchy: the definition is an Algorithm, each named cluster of components is an ordered Procedure, each component is a Pattern, and the sliders, panels, and computed values that drive the definition become Parameters — typed as constant, variable, or emergent — exposed through input and output Interfaces

(Figure 6). The value of this mapping is that it makes a design rule and the parametric definition that must satisfy it mutually addressable. The `ATTRIBUTE_OF` bridge connects a rule atom — for instance, the variable bound by a maximum-height comparison — to the specific parameter that produces the value being checked. A rule can therefore be traced back to the exact slider or computed quantity it constrains, and, conversely, a parameter can report which rules govern it. This two-way correspondence underpins the bidirectional translation between design grammars and parametric definitions on which the lifecycle workflow of section 4 rests: rules authored in natural language are projected onto the parameters of a live definition, and changes to a definition are checked against the rules attached to its parameters. Section 5.3 reports the current implementation status of this bridge honestly: it is schema-conformant but not yet exercised by a populated design-space run.

3.6 Property-graph implementation and OWL semantics

The ontology is specified in OWL 2 DL and its rules in SWRL, but the runtime graph is a Neo4j labelled-property graph, not a triple store — the two are different data models, and the mapping between them cannot be left implicit. A normative, versioned mapping fixes the projection in both directions: every property-graph node label, relationship type, and edge property maps onto a standard term from the W3C OWL and SWRL RDF vocabularies, and the mapping is exercised at reasoning time, not merely declared.

What is projected: Ontograph classes and properties map onto `owl:Class`, `owl:DatatypeProperty`, and `owl:ObjectProperty`; Metagraph Rule, Atom, Var, Literal, and Builtin nodes map onto `swrl:Imp` and its constituent atom types, with the graph's ordered ARG and HAS_BODY/HAS_HEAD edge properties reified as RDF lists so that atom and argument order — which the property graph stores as edge properties, a construct RDF has no direct equivalent for — survives the round trip.

What is lost, deliberately: the per-project domain vocabulary is unioned into the reasoning graph only at reasoning time, kept separate from the static core TBox so that one project's classes cannot leak into another's terminological reasoning; and BuiltinAtom arguments the target OWL reasoner (HermiT) cannot parse are stripped before reasoning, with the stripped count surfaced in the response rather than silently dropped — a defensive step forced by an early failure mode in which an unstripped SWRL builtin crashed the reasoner outright.

Concretely, when a rule is evaluated: the SWRL validator operates natively on the property graph for the quantitative threshold checks that dominate this ontology's rule population (section 3.4); the OWL 2 DL reasoner and the SHACL validator operate on an RDF projection assembled on demand by the isolated reasoner sidecar, which unions the static core TBox, a curated disjointness-axiom overlay, and the live per-project export, so that terminological reasoning never blocks the validation path and a reasoner failure cannot corrupt the graph it reasons about. All three verdicts are written back to the same ValidGraph regardless of which engine produced them, so the evidence trail in section 5 does not distinguish its origin by data model.

The property-graph commitment was made for operational reasons — native support for the write-heavy authoring workflow and a query surface already familiar to the BIM-adjacent tooling ecosystem — not because RDF was found unsuitable; the projection exists precisely so that the OWL semantics the paper claims remain mechanically derivable from the graph that is actually queried, rather than aspirational. Section 6.2 discusses the complementary GQL standardisation argument for the property-graph choice; the present subsection establishes that nothing in the runtime path requires abandoning standard OWL 2 DL semantics to get there.

4. LIFECYCLE INTEGRATION AND AI-FACILITATED WORKFLOW

Although this section describes the runtime that grounds the schema (section 2.1), one of its constructs is schema, not plumbing: the DG ID realises a first-class identity type in the Core band, and the cross-platform mechanism below follows directly from that modelling choice. Cross-platform continuity rests on treating the design object itself as the carrier of identity (Figure 6). Every object serialised into the graph is minted a durable, platform-neutral identity — the DG ID — to which native platform identifiers (a Grasshopper instance GUID, a Revit Uniqueld, an IFC GlobalId, a viewer applicationId) are bound as representations. Separated representations of one conceptual object thus resolve to a single identity within a Design State, and properties computed on one representation become readable, with provenance, from every other: an insulation value computed in the parametric model is exposed on the corresponding BIM element. This realises object-level semantic enrichment — the object accumulates meaning as it moves through tools — rather than the file-level exchange of conventional interoperability, and it is the mechanism by which externalised design intent travels

with the object across the lifecycle.

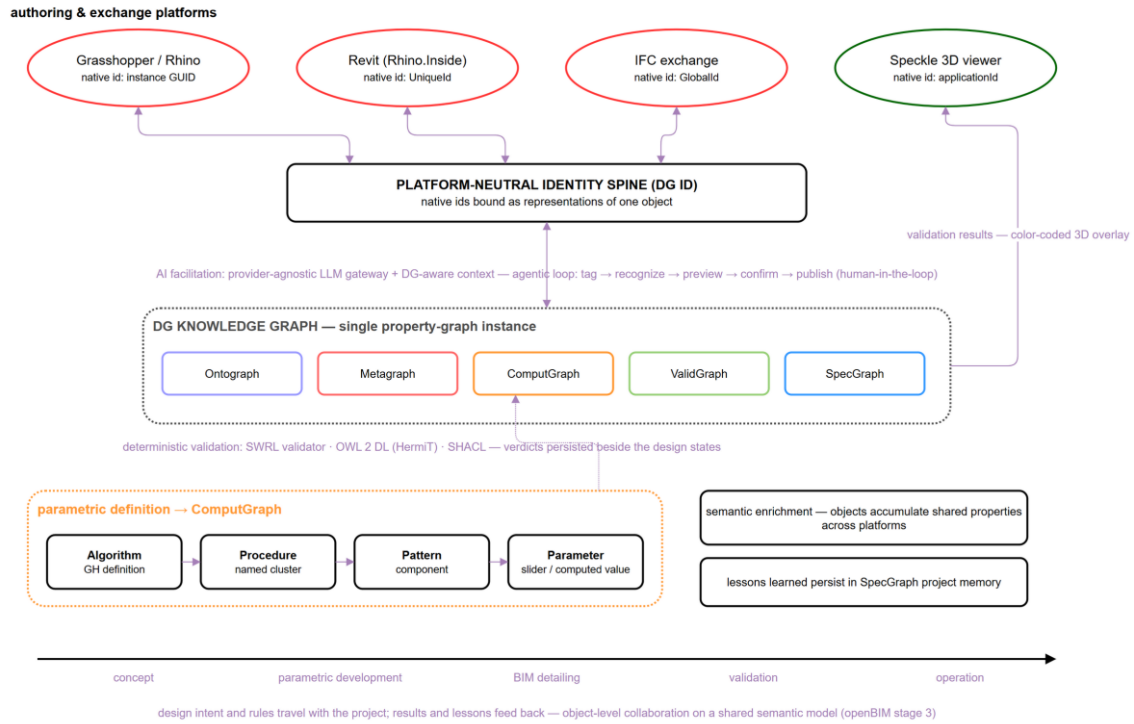


Figure 6: The knowledge graph as the hub of a cross-platform lifecycle: platform representations bound to a platform-neutral identity; AI facilitation above, deterministic validation below; the parametric definition mapped onto the ComputGraph.

The ComputGraph acquires its content through serialisation of the live parametric canvas, following an annotation convention that architects apply as ordinary group labels. Recognition of untagged canvas regions is delegated to a large language model behind a provider-agnostic gateway (Anthropic, OpenAI-compatible, or local models), reflecting the emerging pattern of protocol-mediated agentic BIM interaction (Heimig-Elschner et al., 2026). Every call is prepared by a context assembler — a service that compiles the current ontology catalogue, the SWRL conventions, and a versioned library of parameterised query templates into the model's context — and every generated statement is schema-validated before it reaches the graph, the graph-grounded discipline that distinguishes reliable text-to-Cypher systems (Gusarov et al., 2025). The interaction contract is deliberately conservative: tag → recognise → preview → confirm → publish. The model proposes on-canvas, as temporary groups and annotations; the architect disposes; nothing is written without confirmation, matching the emerging consensus for conversational design agents in architecture (Cheung et al., 2025). On the same substrate, the definition's node-and-wire topology is persisted so that any two snapshots are diffable — the evolution of the generative process itself, not only of its outputs, becomes queryable — and a rule-aware consulting agent answers design questions against the graph, proposing but never executing canvas actions. In parallel, the SpecGraph accumulates decisions, debugging notes, and conventions beside the formal rules they explain, addressing the knowledge-loss problem long documented for design rationale in the AEC industry (Wyke & Lindhard, 2025).

5. ONTOLOGY EVALUATION THROUGH COMPETENCY QUESTIONS

Because this paper concerns the ontology rather than a deployed system, evaluation proceeds competency question by competency question (Table 2): each of the following subsections states the question, gives the Cypher query executed directly against the running graph, and shows the rows the graph actually returned. Section 5.4 complements this with machine-checked terminological evaluation (CQ4) and states what the regression test suite verifies.

5.1 CQ1 — Rule decomposition and grounding

CQ1 asks whether a design rule expressed in natural language can be decomposed into individually addressable, auditable parts and grounded in shared vocabulary. The minimum-building-separation rule (section 3.4) is queried by its full body/head decomposition:

```
MATCH (r:Rule {Rule_Id:'R_BUILDING_MIN_DISTANCE_12_V'})-[m:HAS_BODY|HAS_HEAD]-
>(a:Atom)
OPTIONAL MATCH (a)-[g:ARG]->(x) OPTIONAL MATCH (a)-[:REFERS_TO]->(t)
RETURN type(m), m.order, a.type, t.iri, g.pos, coalesce(x.name, x.lex)
ORDER BY type(m) DESC, m.order, g.pos
```

Table 5: CQ1 result — the seven rows returned by the query above.

Part	Order	Atom type	Grounds (REFERS_TO)	Arg pos	Argument
HAS_BODY	1	ClassAtom	ex:Building	1	?b
HAS_BODY	2	DataPropertyAtom	ex:hasDistanceM	1	?b
HAS_BODY	2	DataPropertyAtom	ex:hasDistanceM	2	?d
HAS_BODY	3	BuiltinAtom	swrlb:lessThan	1	?d
HAS_BODY	3	BuiltinAtom	swrlb:lessThan	2	12
HAS_HEAD	1	DataPropertyAtom	ex:violatesMinDistance	1	?b
HAS_HEAD	1	DataPropertyAtom	ex:violatesMinDistance	2	true

The seven returned rows are exactly the rule's three body atoms (a ClassAtom grounded in ex:Building, a DataPropertyAtom grounded in ex:hasDistanceM, and a BuiltinAtom grounded in swrlb:lessThan) and one head atom (grounded in ex:violatesMinDistance), each with its ordered arguments. Figure 7a renders the same rows as a populated subgraph. Every atom is individually addressable and re-queryable, the rule carries its natural-language description alongside its structured identifier and SWRL string side by side, and every REFERS_TO edge grounds an atom in a class or property that the Ontograph, not the rule, owns — so the same vocabulary term is never redefined per rule. The result answers CQ1 directly: decomposition and grounding are not aspirational, they are what the query returns.

5.2 CQ2 — Per-object validation verdicts

CQ2 asks whether the current design state of a scheme satisfies the constraints defined for it, with an individually addressable verdict per object. A persisted validation run (v8smoke-run-001) checks the 75 m maximum-height rule against eight Building objects; the entities and verdicts it recorded are read back with:

```
MATCH (run:ValidationRun {runId:'v8smoke-run-001'})-[:HAS_ENTITY]->(e:ValidationEntity)
RETURN e.dgEntityId, e.ruleId, e.status ORDER BY e.dgEntityId
```

Table 6: CQ2 result — the eight per-object verdicts returned by the query above.

Object	Rule	Verdict
building1	R_URB_HEIGHT_MAX_75_V	passed
building2	R_URB_HEIGHT_MAX_75_V	failed
building3	R_URB_HEIGHT_MAX_75_V	passed
building4	R_URB_HEIGHT_MAX_75_V	failed
building5	R_URB_HEIGHT_MAX_75_V	passed
building6	R_URB_HEIGHT_MAX_75_V	passed
building7	R_URB_HEIGHT_MAX_75_V	failed
building8	R_URB_HEIGHT_MAX_75_V	passed

The eight returned rows record three violations of eight objects, each individually addressable and traceable back through `HAS_ENTITY` to the Run node and, through the run's persisted `statePayloadJson`, to the `ObjState/ParamState/PropState` snapshot it was evaluated against (Figure 7b). Because the violation-pattern semantics makes the absence of a recorded violation a positive statement of compliance only once the graph is known to be fully populated (section 6.1), the run additionally records `SendStatus`, confirming that all eight verdicts, not a partial set, were published to the 3D viewer. The same mechanism composes across constraint categories without change: the schema's rule catalogue already includes minimum-separation (`R_BUILDING_MIN_DISTANCE_12_V`, section 3.4), minimum floor-plate area (`R_LIVINGUNIT_MIN_AREA_30_V`), and minimum street width (`R_STREET_MIN_WIDTH_18_V`) rules following the identical decomposition pattern; a single validation run can attach any number of them to the same Design State, and the answer to CQ2 does not depend on how many are attached at once.

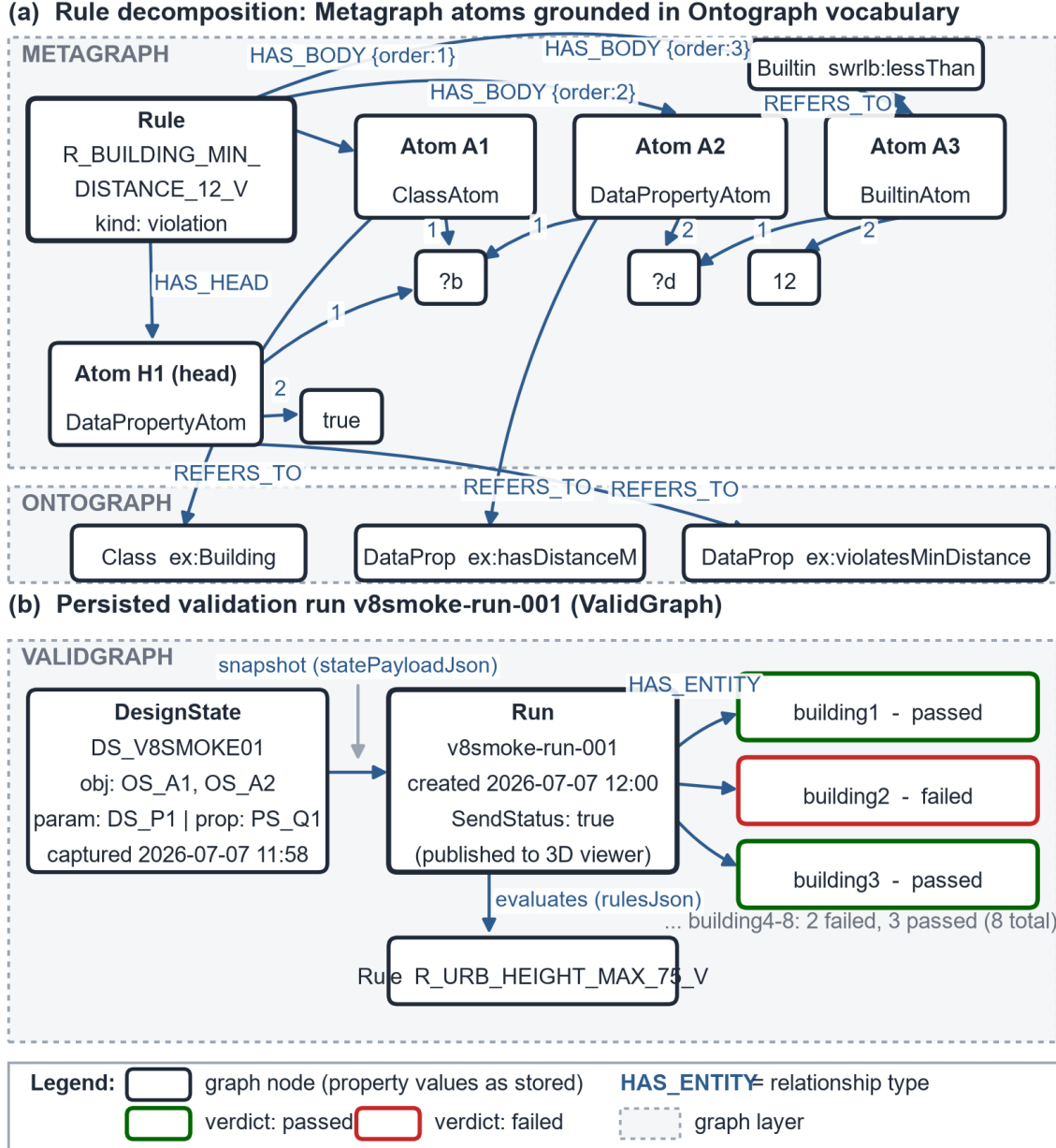


Figure 7: A populated subgraph read back from the running system — (a) the minimum-distance rule's full atom decomposition and grounding (CQ1, §5.1); (b) the persisted validation run and its per-object verdicts (CQ2, §5.2), with real property values as stored.

5.3 CQ3 — Tracing a rule to its governing parameter

CQ3 asks whether a planning-constraint rule can be traced to the specific parametric quantity it constrains, and, conversely, whether a parameter can report which rules govern it — the mechanism a design-space-exploration workflow would filter candidates with. The schema defines this as the `ATTRIBUTE_OF` bridge, a domain-Atom, range-Parameter relationship (section 3.2) that would be queried as:

```
MATCH (r:Rule {Rule_Id:$ruleId})-[:HAS_BODY]->(a:Atom)-[:ATTRIBUTE_OF]->(p:Parameter)
RETURN r.Rule_Id, a.Atom_Id, p.parameterName
```

Unlike CQ1 and CQ2, this query currently returns no rows: the `ATTRIBUTE_OF` bridge is implemented in the schema and exercised by the property-graph-to-OWL projection (section 3.6), but the shipped case-study data does not yet include a populated `ComputGraph` run with parameters bound to rule atoms, so CQ3 is answered at the mechanism level rather than the data level. This is a genuine gap in the evidence, not a hedge: the honest status is that the bidirectional rule-parameter trace described in section 3.5 is implemented and schema-conformant but not yet demonstrated against a populated design-space run. Closing it — populating a `ComputGraph` instance, attaching `ATTRIBUTE_OF` edges during rule authoring, and re-running the query above until it returns rows — is the immediate next step identified for future work (section 7), ahead of the design-space-exploration application the schema was built to support.

5.4 CQ4 and implementation evidence

CQ4 asks whether the schema itself is terminologically consistent and structurally complete under its own integrity constraints — the question the competency questions above presuppose an affirmative answer to every time they run. Two machine-checked reasoning services answer it directly. The OWL 2 DL reasoner (HermiT), run over the hybrid TBox described in section 3.6, reports the schema consistent with zero unsatisfiable classes; the same run resolves 65 subclass axioms, 101 property domains, 140 property ranges, and 2 disjointness axioms from a curated overlay, after stripping 5 BuiltinAtom rules the reasoner cannot parse natively (section 3.6). The SHACL validator, run over the same project's structural shapes, reports full conformance: zero violations, warnings, or informational findings. Both checks are reproducible against the live system (POST /reason/consistency and POST /shacl/validate on the reasoner sidecar) rather than one-off manual audits.

The schema was additionally self-audited against the pitfall catalogue used by the OOPS! ontology-evaluation tool (Poveda-Villalón et al., 2014), the recognised evaluation instrument for this class of check. Every one of the 62 core classes carries both an `rdfs:label` and an `rdfs:comment` — no missing-annotation pitfall. Ten of the schema's 111 object and datatype properties lack an explicit `rdfs:domain` and two lack an explicit `rdfs:range` — a genuine, acknowledged gap rather than a claimed clean bill of health; closing it is tracked alongside the future-work items of section 7. Class disjointness is deliberately not embedded in the core TBox but supplied through a separate, curated overlay so the core stays reasoner-fast; the overlay currently declares two disjointness axioms, and their effect is visible directly in the consistency-check axiom counts above.

Implementability is further evidenced by the co-developed runtime. The ontology has passed through three formalisation versions, with the core band and layer partition unchanged since the previous version — an indication that the conceptual structure has converged while the property vocabulary continues to grow additively. A regression test suite spans four service boundaries: `OntoGraph/Metagraph/ValidGraph/SpecGraph` model round-trips and SWRL rule parsing in the authoring-tool plugin; `Design State (ObjState/ParamState/PropState)` serialisation, parameter binding, and reinstatement in the same plugin; `canvas-to-ComputGraph` annotation parsing and connector-heartbeat contracts; and OWL/SHACL export, consistency reasoning, and validation-publish integrity in the graph data service and reasoner sidecar. Every schema change is required to propagate across the ontology file, the Cypher templates, the LPG-OWL mapping, and the SHACL shapes before it is considered complete, a checklist enforced in the project's own contribution documentation, which is what the regression suite exists to catch when that propagation is incomplete.

6. DISCUSSION

6.1 Expressiveness and limitations

The current schema encodes a subset of OWL 2 + SWRL expressiveness sufficient for quantitative threshold constraints — minimum, maximum, range, equality, cardinality — through three atom types (`ClassAtom`,

DataPropertyAtom, BuiltinAtom). It does not yet support disjunctive rules, complex class expressions, or temporal constraints, which are common in building energy codes and fire-safety regulations; extending the schema toward these patterns, most likely through a connector node type representing logical operators between atoms, is a priority for future work. The violation-pattern semantics also operates under an open-world assumption: if no building object is present in the graph, a height rule will not fire, irrespective of whether the constraint is satisfied — counterintuitive for practitioners expecting closed-world compliance semantics. The mitigation is twofold: the graph is pre-populated with all model objects before rule evaluation, so that the absence of a violation genuinely indicates compliance rather than an empty graph, and the SHACL layer closes the world for structural completeness — a shape violation reports exactly the missing population that would otherwise silence a rule.

6.2 Positioning and scalability

Table 7 positions the proposed ontology against comparison points along the dimensions that matter for design-intent persistence: whether rules persist as first-class artifacts, whether they are independently auditable, the data-transformation burden imposed on the authoring-tool side, alignment with open standards, and whether verdicts are deterministically re-executable.

Table 7: Positioning against comparison approaches.

Approach	Rules persist as artifacts	Independently auditable	Transformation burden	Standards alignment	Deterministic re-execution
IfcOWL + SPARQL (Pauwels et al., 2017)	Yes, as RDF triples	Yes	High — full IfcOWL serialisation	OWL/SPARQL	Yes
Proprietary rule engines	Yes, in a closed format	No — opaque to inspection	Low–moderate	None	Vendor-dependent
SHACL-only validation (Hagedorn et al., 2023)	Shapes persist, express structure not intent	Yes, for structural checks	Moderate	SHACL/RDF	Yes
LLM-first compliance checking (Iversen & Huang, 2026; Madireddy et al., 2025)	No — lives in a prompt-response cycle	No — implicit in the context window	Low	None claimed	No — reinterpretation may vary
DG ontology (this paper)	Yes — decomposed SWRL atoms in the Metagraph	Yes — atom-by-atom, versioned	Low — no full IfcOWL serialisation required	OWL 2 DL / SWRL / SHACL / GQL path	Yes

The two approaches converge rather than compete: recent agentic architectures ground LLM interaction in property graphs and protocol-mediated tool access (Gusarov et al., 2025; Heimig-Elschner et al., 2026), and the DG system embodies this synthesis — LLMs where they are strong, deterministic validation wherever verdicts matter. Taken together, these mechanisms address the defining requirements of openBIM maturity stage 3: a single shared semantic model, object-level rather than file-level collaboration, lifecycle-wide information management, and open-standards alignment (Palihakkara et al., 2025; buildingSMART, 2025, 2026).

Regarding scalability, project isolation via a property filter within a single graph suits the pilot scale defined in section 3.2 but should be revisited for multi-tenant settings, where sharding or per-project databases may be preferable; city-scale models and national regulatory libraries will additionally require query-indexing strategies. The property-graph commitment is future-proofed by standardisation: the GQL standard (ISO/IEC, 2024) defines graph types as a schema layer for property graphs, providing a vendor-neutral migration path for the normative schema beyond any single database product.

7. CONCLUSION

This paper set out to answer a question the LLM-centric compliance-checking wave leaves open: once a rule has been interpreted from natural language, where does it live? The answer developed here is a schema, not a system: the Design Grammar ontology gives the interpreted rule a durable, queryable home — decomposed into individually addressable SWRL atoms, grounded in a shared vocabulary that is itself machine-checked for

consistency, and evaluated under a violation-pattern encoding whose absence-of-violation verdict is made trustworthy by explicit population discipline rather than assumed. Three of the paper's four competency questions are answered directly against the running graph with the query and the returned rows shown (section 5); the fourth is answered honestly rather than convincingly — the rule-to-parameter trace is implemented and schema-conformant but not yet exercised by a populated design-space run, and closing that gap is the paper's most concrete piece of unfinished business.

What was not known before this paper is not that design intent can be modelled — Gero's Function has been available for three decades — but that it can be modelled as a graph-resident, externally addressable construct without becoming either an unauditable prompt artifact or a closed proprietary rule format: the object-centred FBS core, the property-graph-to-OWL projection that keeps that core provably consistent, and the identity-carrier mechanism that lets one conceptual object's semantic properties survive a platform change are what make that claim checkable rather than aspirational.

Future work follows directly from where the evidence in section 5 runs out: (1) populate a design-space-exploration case study that exercises the `ATTRIBUTE_OF` bridge end to end, closing CQ3; (2) extend the rule schema to disjunctive and temporal constraints, which the current three-atom-type encoding does not support (section 6.1); (3) close the ten missing property domains and two missing ranges identified in the section 5.4 self-audit; and (4) deepen the standards bridges toward full topological reasoning and requirement-driven exchange workflows (IfcOWL/IDS). The ontology, its extension modules, and the supporting artifacts will be released as open source under the terms stated below, to support community adoption and comparative evaluation.

AVAILABILITY

The core ontology module, its three extension modules (standards, BOT, Topologic), the SHACL shape set, and the normative property-graph-to-OWL mapping documentation described in this paper are maintained as a versioned project and will be deposited at an open, citable archive (Zenodo or equivalent) with an assigned DOI and an explicit open licence (Creative Commons Attribution 4.0, matching this paper's own licence) upon acceptance; the namespace currently in use (<http://example.org/design-grammar#>) is a development placeholder and will be replaced by a resolving, permanent namespace at the same time, consistent with linked-data best practice. The runtime services that exercise the schema — the graph data service, the reasoner sidecar, and the Grasshopper authoring plugin — are versioned in the same repository and will be released under the same licence.

ACKNOWLEDGEMENTS

This work is supported by national funds through FCT — Foundation for Science and Technology under grant agreement 2024.03763.BD attributed to the first author, and by the R2U Technologies | modular systems project (ref.: C644876810-00000019) funded by PRR — Plano de Recuperação e Resiliência — and by European Funds Next Generation EU. Additional support was provided through the Multiannual Funding of Lab2PT (UID/04509), FCT R&D Unit ISISE (UID/4029/2025; UID/PRR/04029/2025), and Associate Laboratory ARISE (LA/P/0112/2020).

REFERENCES

- Beirão, J. N., Duarte, J. P., & Stouffs, R. (2011). Creating Lisbon: Urban design with a city grammar. In *Proceedings of the 2011 Symposium on Simulation for Architecture and Urban Design (SimAUD)*.
- buildingSMART International. (2025). *Global openBIM mandates: 2025 edition*. https://www.buildingsmart.org/wp-content/uploads/2025/03/IFC-Mandate_2025.pdf
- buildingSMART International. (2026). *Embracing challenges and opportunities of using openBIM for construction: Industry insight report*. https://www.buildingsmart.org/wp-content/uploads/2026/03/Construction-openBIM_Industry_Insight_bSI_final.pdf
- Chen, N., Lin, X., Jiang, H., & An, Y. (2024). Automated building information modeling compliance check through a large language model combined with deep learning and ontology. *Buildings*, 14(7), 1983. <https://doi.org/10.3390/buildings14071983>

- Cheung, L. H., Lombardi, D., & Di Marco, G. (2025). Agentic, multimodal large language model (LLM) with behavioural parameters and self-iterative capabilities for conversational architectural design processes. *Architectural Science Review*, 1–21. <https://doi.org/10.1080/00038628.2025.2586655>
- Dorst, K. (2011). The core of 'design thinking' and its application. *Design Studies*, 32(6), 521–532. <https://doi.org/10.1016/j.destud.2011.07.006>
- Dorst, K., & Cross, N. (2001). Creativity in the design process: Co-evolution of problem–solution. *Design Studies*, 22(5), 425–437. [https://doi.org/10.1016/S0142-694X\(01\)00009-6](https://doi.org/10.1016/S0142-694X(01)00009-6)
- Duarte, J. P. (2005). Towards the mass customization of housing: The grammar of Siza's Malagueira houses. *Environment and Planning B: Planning and Design*, 32(3), 347–380. <https://doi.org/10.1068/b3144>
- Eastman, C., Lee, J.-m., Jeong, Y.-s., & Lee, J.-k. (2009). Automatic rule-based checking of building designs. *Automation in Construction*, 18(8), 1011–1033. <https://doi.org/10.1016/j.autcon.2009.07.002>
- Ermolenko, E., Figueiredo, B., & Azenha, M. (2026). Management of the design knowledge core in a cross-platform environment of the BIM project. In *Proceedings of the ptBIM 2026 Conference*. University of Minho.
- Fortune Business Insights. (2026). *Building information modeling (BIM) market size, share & industry analysis, with forecast to 2034* (Report No. 102986). <https://www.fortunebusinessinsights.com/building-information-modelling-software-market-102986>
- Fu, K., Murphy, J., Yang, M., Otto, K., Jensen, D., & Wood, K. (2015). Design-by-analogy: Experimental evaluation of a functional analogy search methodology for concept generation improvement. *Research in Engineering Design*, 26(1), 77–95. <https://doi.org/10.1007/s00163-014-0186-4>
- Gero, J. S. (1990). Design prototypes: A knowledge representation schema for design. *AI Magazine*, 11(4), 26–36.
- Gero, J. S. (1992). What's in a design case and how it can be used. In *AID'92 Workshop on Case-Based Design Systems* (pp. 27–28).
- Gero, J. S., & Kannengiesser, U. (2004). The situated function–behaviour–structure framework. *Design Studies*, 25(4), 373–391. <https://doi.org/10.1016/j.destud.2003.10.010>
- Gusarov, A., Volkova, A., Khrulkov, V., Kuznetsov, A., Maslov, E., & Oseledets, I. (2025). *Multi-agent GraphRAG: A text-to-Cypher framework for labeled property graphs*. arXiv. <https://doi.org/10.48550/arXiv.2511.08274>
- Hagedorn, P., Pauwels, P., & König, M. (2023). Semantic rule checking of cross-domain building data in information containers for linked document delivery using the shapes constraint language. *Automation in Construction*, 156, 105106. <https://doi.org/10.1016/j.autcon.2023.105106>
- Heimig-Elschner, T., Du, C., Scheuven, A., Borrmann, A., & Beetz, J. (2026). *A modular reference architecture for MCP-servers enabling agentic BIM interaction*. arXiv. <https://arxiv.org/abs/2601.00809>
- ISO/IEC. (2024). *ISO/IEC 39075:2024 — Information technology — Database languages — GQL*. International Organization for Standardization.
- Iversen, O., & Huang, L. (2026). Leveraging large language models for BIM-based automated compliance checking. *Automation in Construction*, 182, 106707. <https://doi.org/10.1016/j.autcon.2025.106707>
- Jabi, W., & Chatzivasileiadi, A. (2021). Topologic: Exploring spatial reasoning through geometry, topology, and semantics. In *Formal Methods in Architecture* (pp. 277–285). Springer. https://doi.org/10.1007/978-3-030-57509-0_25
- Janowicz, K., Haller, A., Cox, S. J. D., Le Phuoc, D., & Lefrançois, M. (2019). SOSA: A lightweight ontology for sensors, observations, samples, and actuators. *Journal of Web Semantics*, 56, 1–10. <https://doi.org/10.1016/j.websem.2018.06.003>
- Lin, C.-J. (2016). Design criteria modeling — Use of ontology-based algorithmic modeling to represent architectural design criteria at the conceptual design stage. *Computer-Aided Design and Applications*, 13(4), 549–557. <https://doi.org/10.1080/16864360.2015.1131551>

- Madireddy, S., Gao, L., Din, Z. U., Kim, K., Senouci, A., Han, Z., & Zhang, Y. (2025). Large language model-driven code compliance checking in building information modeling. *Electronics*, 14(11), 2146. <https://doi.org/10.3390/electronics14112146>
- Mubarak, K. (2004). *Case-based reasoning for design composition in architecture* [Doctoral dissertation, Carnegie Mellon University].
- Noy, N. F., & McGuinness, D. L. (2001). *Ontology development 101: A guide to creating your first ontology*. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- Palihakkara, A. D., Osorio-Sandoval, C. A., & Tizani, W. (2025). A review of the application of ontologies and semantic web for building information modelling and digital twins based construction management. *Journal of Information Technology in Construction (ITcon)*, 30, 1208–1243. <https://doi.org/10.36680/j.itcon.2025.049>
- Pauwels, P., & Terkaj, W. (2016). EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology. *Automation in Construction*, 63, 100–133. <https://doi.org/10.1016/j.autcon.2015.12.003>
- Pauwels, P., Zhang, S., & Lee, Y. C. (2017). Semantic web technologies in AEC industry: A literature overview. *Automation in Construction*, 83, 145–165. <https://doi.org/10.1016/j.autcon.2017.03.009>
- Poon, J., & Maher, M. L. (1997). Co-evolution and emergence in design. *Artificial Intelligence in Engineering*, 11(3), 319–327. [https://doi.org/10.1016/S0954-1810\(96\)00047-7](https://doi.org/10.1016/S0954-1810(96)00047-7)
- Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M., & García-Castro, R. (2022). LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111, 104755. <https://doi.org/10.1016/j.engappai.2022.104755>
- Poveda-Villalón, M., Gómez-Pérez, A., & Suárez-Figueroa, M. C. (2014). OOPS! (OntOlogy Pitfall Scanner!): An on-line tool for ontology evaluation. *International Journal on Semantic Web and Information Systems*, 10(2), 7–34. <https://doi.org/10.4018/ijswis.2014040102>
- Rasmussen, M. H., Lefrançois, M., Schneider, G. F., & Pauwels, P. (2020). BOT: The building topology ontology of the W3C linked building data community group. *Semantic Web*, 12(1), 143–161. <https://doi.org/10.3233/SW-200385>
- RIBA. (2020). *RIBA Plan of Work 2020 overview*. Royal Institute of British Architects.
- Sack, H. (2019). *Knowledge engineering with semantic web technologies: Ontological engineering* [MOOC]. openHPI.
- Sacks, R., Brilakis, I., Pikas, E., Xie, H. S., & Girolami, M. (2020). Construction with digital twin information systems. *Data-Centric Engineering*, 1, e14. <https://doi.org/10.1017/dce.2020.16>
- Schön, D. A. (1992). Designing as reflective conversation with the materials of a design situation. *Research in Engineering Design*, 3(3), 131–147. <https://doi.org/10.1007/BF01580516>
- Stiny, G., & Gips, J. (1972). Shape grammars and the generative specification of painting and sculpture. In C. V. Freiman (Ed.), *Information Processing 71* (pp. 1460–1465). North-Holland.
- Suárez-Figueroa, M. C., Gómez-Pérez, A., & Fernández-López, M. (2015). The NeOn methodology for ontology engineering. In M. C. Suárez-Figueroa, A. Gómez-Pérez, E. Motta, & A. Gangemi (Eds.), *Ontology engineering in a networked world* (pp. 9–34). Springer. https://doi.org/10.1007/978-3-642-24794-1_2
- Wyke, S., & Lindhard, S. M. (2025). Understanding design rationale and intent through natural language processing analysis: A search for consensus. *Journal of Information Technology in Construction (ITcon)*, 30, 631–649. <https://doi.org/10.36680/j.itcon.2025.026>
- Zhou, Y., Braun, A., & Borrmann, A. (2022). Knowledge graph-based automated code compliance checking for building design. In *Proceedings of the European Conference on Computing in Construction (EC3)*. <https://doi.org/10.35490/EC3.2022.188>